

Universidad Andrés Bello
Magíster en Ciencia de Datos

Proyecto final integrador Fase 4
Análisis de condiciones operacionales y fallas con AI4I 2020
MCDI500 Programación para la Ciencia de Datos

Estudiante: Alexander Sepúlveda Ormazábal

Grupo: 9

Docente responsable: Omar Salinas Silva

Fecha: 4 de octubre de 2026

github.com/alexander-seor/proyecto-grupo9-mcdi500

Índice

1 Introducción y contextualización	1
2 Definición de la problemática y objetivos	1
3 Aplicación de herramientas científicas	2
4 Reproducibilidad técnica.....	2
5 Control de versiones	2
6 Documentación del proceso	2
7 Repositorio GitHub actualizado	2
8 Diseño de soluciones algorítmicas eficientes.....	3
8.1 Codificación funcional.....	3
8.2 Preprocesamiento y transformación del dataset.....	3
8.3 Validación técnica y verificación del código.....	3
8.4 Eficiencia y optimización	3
8.5 Diseño estructurado del código	3
8.6 Notebook final ejecutado y documentado	3
9 Implementación de código modular y robusto.....	3
9.1 Programación orientada a objetos.....	3
9.2 Documentación de arquitectura	3
10 Construcción de visualizaciones efectivas	4
10.1 Preprocesamiento para visualización	4
10.2 Visualizaciones analíticas	4
11 Metodología del desarrollo técnico	7
12 Validación y reflexión técnica	7
13 Resultados.....	7
14 Discusión	7
15 Conclusiones	7
16 Trazabilidad de mejoras.....	8
17 Presentación audiovisual	8
18 Bibliografía	8

1 Introducción y contextualización

Las fallas inesperadas interrumpen la operación y aumentan las necesidades de mantenimiento. El proyecto estudia AI4I 2020, un conjunto sintético con 10.000 registros de temperatura, velocidad de rotación, torque, desgaste y tipo de producto, además de una variable binaria de falla (Matzka, 2020). La utilidad del análisis consiste en reconocer condiciones que merecen atención antes de avanzar a una evaluación predictiva formal.

El cierre integra las cuatro fases. F1 delimitó el problema y el entorno; F2 preparó los datos; F3 reorganizó el pipeline con funciones, clases y módulos; F4 recuperó una versión interpretable, construyó las visualizaciones y comprobó los hallazgos en las particiones de entrenamiento y prueba. El mantenimiento predictivo requiere distinguir los patrones descriptivos del rendimiento de un modelo, especialmente cuando la clase de interés es poco frecuente (Phan et al., 2022).

2 Definición de la problemática y objetivos

La pregunta central es: ¿qué condiciones operacionales están asociadas con la ocurrencia de fallas y en qué medida permiten identificar situaciones de riesgo? El objetivo general es analizar esa relación, identificar posibles patrones y evaluar posteriormente la capacidad de anticipación. En esta entrega se completa la caracterización descriptiva; la evaluación predictiva queda fuera del alcance porque no se entrenó un clasificador.

Los objetivos específicos abordados fueron:

1. Caracterizar las variables operacionales, el tipo de producto y la variable de falla.
2. Conservar un flujo reproducible desde el archivo original hasta los productos de F4.
3. Explorar las tasas de falla por tipo, quintiles e interacción de condiciones.
4. Comprobar en prueba el perfil definido con los datos de entrenamiento.

Alcance. El análisis considera las seis variables predictoras definidas en F1 y la variable Machine failure. Los identificadores y los cinco modos de falla permanecen excluidos para evitar fuga de información.

Restricciones. AI4I 2020 es sintético, contiene solo 339 fallas y no representa una planta industrial específica. El plazo del curso limita el cierre a un análisis descriptivo.

Supuestos. Las unidades y definiciones publicadas por UCI se consideran válidas. Los registros se interpretan como observaciones independientes y los parámetros guardados por F3 corresponden al entrenamiento ejecutado.

3 Aplicación de herramientas científicas

NumPy controla la semilla y operaciones numéricas; pandas carga, agrupa y valida tablas; scikit-learn sustenta la división, One Hot y RobustScaler; Matplotlib y seaborn producen las figuras. Jupyter integra código, resultados y explicación en una secuencia ejecutable. Git registra la evolución del repositorio.

Herramienta	Versión	Uso principal
Python	3.13.0	Ejecución del proyecto
NumPy	2.5.3	Semilla y operaciones numéricas
pandas	3.0.5	Tablas, qcut y validación
Matplotlib	3.11.2	Composición de figuras
seaborn	0.13.2	Mapa de calor
scikit-learn	1.9.1	División y transformaciones

Tabla 1. Entorno científico registrado en la ejecución de F4

4 Reproducibilidad técnica

El notebook confirma que el intérprete pertenece a .venv, declara la semilla 42 y muestra las versiones. Las rutas se construyen desde la raíz del repositorio. requirements.txt fija las dependencias y los metadatos guardan la fecha, las entradas y sus huellas SHA-256. La ejecución final contiene 20 celdas de código numeradas del 1 al 20, sin errores.

5 Control de versiones

El historial contiene cambios separados para F1, F2 y F3. Entre los registros verificables se encuentran ed3da7b para la definición de F1, be64efc para el preprocesamiento de F2, 858ba05 para la modularización y eficiencia de F3 y 5805bf0 para su documentación. El desarrollo es individual y no atribuye contribuciones a integrantes inexistentes.

6 Documentación del proceso

Cada fase conserva notebooks, diccionarios, parámetros, metadatos y productos procesados. El notebook de F4 añade tablas de respaldo para cada cifra, un mapa entre objetivos y figuras y una tabla de mejoras. README.md explica el orden de ejecución y F4/README.md describe las entradas, salidas y limitaciones.

7 Repositorio GitHub actualizado

La rama main organiza F1 a F4 y conserva el archivo original separado de los datos procesados. F4 contiene el notebook integrador, la copia interpretable, ocho tablas de apoyo, metadatos y cuatro imágenes. El repositorio se encuentra en github.com/alexander-seor/proyecto-grupo9-mcdi500.

8 Diseño de soluciones algorítmicas eficientes

8.1 Codificación funcional

Las funciones de carga, separación y medición reciben parámetros explícitos y devuelven resultados verificables. La alternativa funcional de F3 sirve como referencia para comprobar que la organización orientada a objetos no modifica el contenido.

8.2 Preprocesamiento y transformación del dataset

F2 excluye UDI, Product ID y los modos TWF, HDF, PWF, OSF y RNF; conserva Type y cinco mediciones; divide 80/20 con estratificación; ajusta One Hot y RobustScaler solo con entrenamiento; aplica los parámetros a prueba y mantiene Machine failure separada. No se imputan ni eliminan filas porque el archivo no contiene nulos ni duplicados exactos.

8.3 Validación técnica y verificación del código

F3 comprueba un caso normal de cinco filas, un caso límite de una fila y dos excepciones: uso del pipeline antes del ajuste y categoría desconocida. También verifica con `assert_frame_equal` que las salidas de F3 coincidan con F2. F4 vuelve a comprobar las 10.000 observaciones contra F1 después de recuperar unidades y categorías.

8.4 Eficiencia y optimización

Implementación	Tiempo medio	Tiempo mínimo	Memoria pico
Funcional	0,0133 s	0,0115 s	0,663 MiB
POO modular	0,0373 s	0,0352 s	1,239 MiB

Tabla 2. Comparación de tiempo y memoria guardada por F3

La versión funcional fue más rápida en 1.000, 5.000, 10.000 y 20.000 filas. La razón POO/funcional varió entre 2,12 y 4,10. Ambas alternativas muestran un crecimiento compatible con $O(n)$, aunque los tiempos breves presentan variación. El proyecto conserva la arquitectura modular por su mantenibilidad y propone reducir copias y agrupar las columnas numéricas como mejora futura.

8.5 Diseño estructurado del código

La cohesión es alta porque `carga.py`, `transformadores.py`, `pipeline.py`, `fabrica.py` y `medicion.py` tienen responsabilidades específicas. El acoplamiento es bajo porque Pipeline recorre una lista de objetos mediante `ajustar` y `transformar`, sin depender de la implementación interna de cada paso.

8.6 Notebook final ejecutado y documentado

F4 integra los cuatro productos de transición, la evidencia de pruebas y eficiencia, la reconstrucción interpretable, el análisis, las figuras y la verificación final. Cada bloque de código tiene una explicación previa y todas las salidas necesarias quedan visibles.

9 Implementación de código modular y robusto

9.1 Programación orientada a objetos

Transformador define el contrato común; CodificadorOneHot y EscaladorRobusto heredan ese comportamiento y aplican tareas distintas. El pipeline usa polimorfismo al llamar los mismos métodos en cada objeto. Los atributos `_parametros` y `_ajustado` encapsulan el estado y las guardas impiden transformar antes de ajustar.

9.2 Documentación de arquitectura

FabricaTransformadores concentra la creación de objetos según el tipo de variable. Este patrón Factory permite cambiar la configuración sin reescribir el orquestador. Los módulos incluyen docstrings, parámetros explícitos y excepciones con mensajes comprensibles.

10 Construcción de visualizaciones efectivas

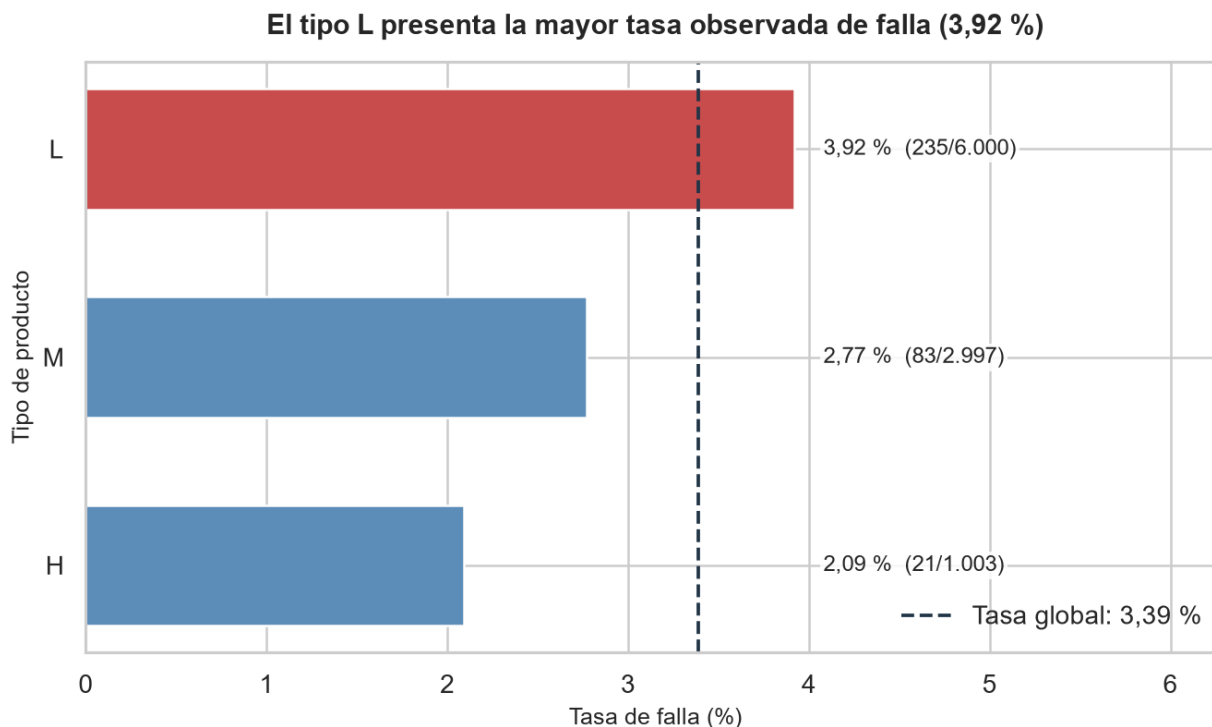
10.1 Preprocesamiento para visualización

Los CSV de F3 son apropiados para algoritmos, pero sus valores numéricos están escalados y Type aparece en tres columnas. F4 recupera cada medición con $\text{original} = \text{escalado} \times \text{IQR} + \text{mediana}$ y reconstruye H, L o M desde la única columna One Hot activa. La copia mantiene una columna de partición y coincide con las siete variables analíticas de F1.

Partición	Grupo	Fallas / registros	Tasa
Entrenamiento	Perfil combinado	79 / 273	28,94 %
Entrenamiento	Resto	192 / 7.727	2,48 %
Prueba	Perfil combinado	22 / 80	27,50 %
Prueba	Resto	46 / 1.920	2,40 %

Tabla 3. Validación descriptiva del perfil combinado por partición

10.2 Visualizaciones analíticas

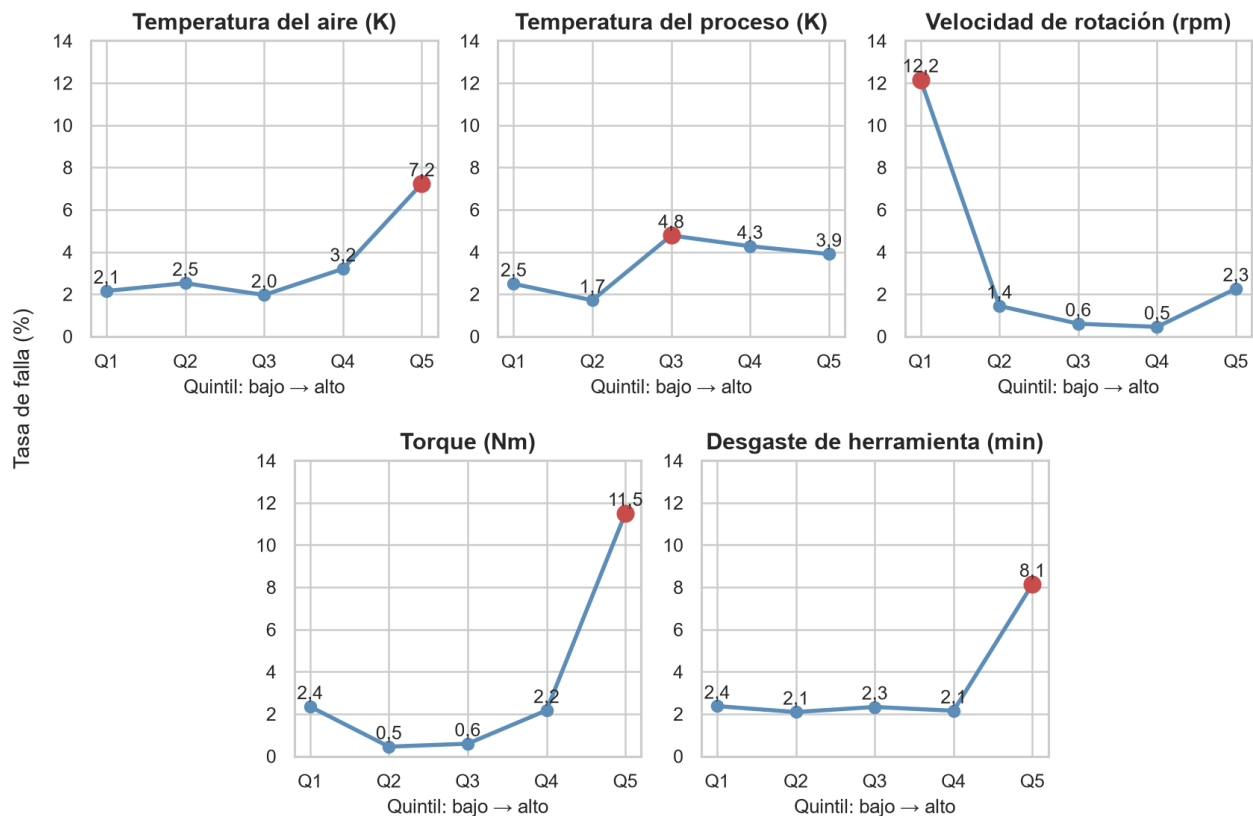


Fuente: Elaboración propia a partir de AI4I 2020 (Matzka, 2020).

Figura 1. El tipo L presenta la mayor tasa observada de falla

La Figura 1 entrega el contexto categórico. L registra 235 fallas entre 6.000 observaciones y una tasa de 3,92 %, superior a la tasa global de 3,39 %. M alcanza 2,77 % y H 2,09 %. Los tamaños de los grupos son distintos y la comparación no demuestra causalidad.

Las tasas más altas aparecen con velocidad baja (12,2 %) y torque alto (11,5 %)

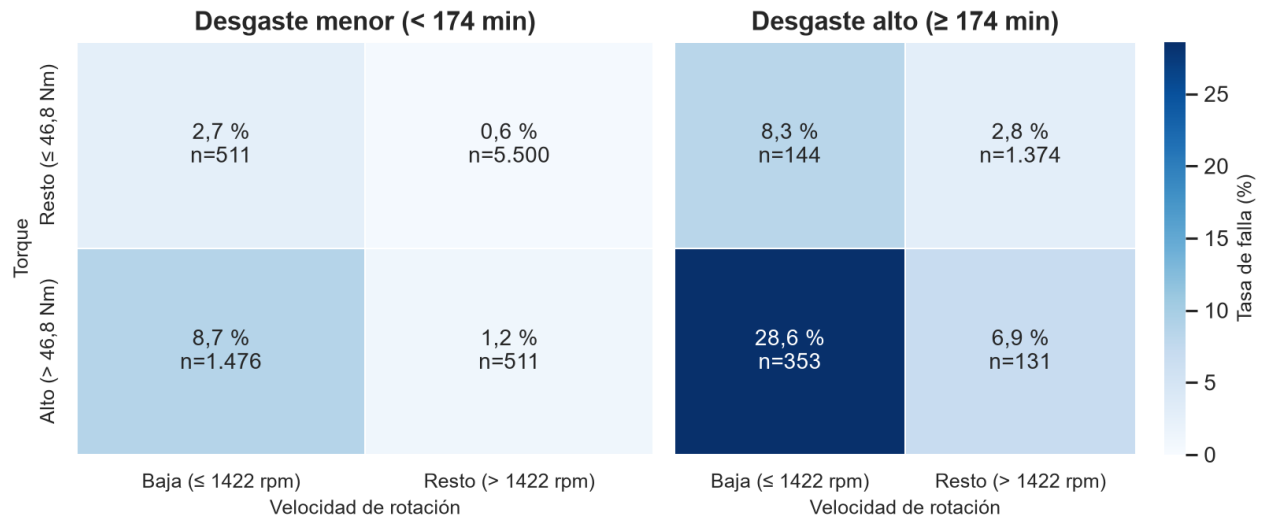


Fuente: elaboración propia a partir de AI4I 2020 (Matzka, 2020). Q1 representa valores bajos y Q5 valores altos; los límites están en la tabla de respaldo.

Figura 2. Las tasas más altas por quintil aparecen con velocidad baja y torque alto

La Figura 2 contrasta las cinco variables en grupos de tamaño parecido. La velocidad alcanza 12,16 % en Q1 y el torque 11,50 % en Q5. El desgaste y la temperatura del aire también aumentan en Q5, mientras la temperatura del proceso no sigue un recorrido creciente. Los intervalos se calcularon con `pandas.qcut` y describen este conjunto; no son límites industriales (`pandas development team`, 2026).

**Con torque alto y velocidad baja, la tasa alcanza 28,6 %
cuando el desgaste es alto**



Fuente: elaboración propia a partir de AI4I 2020 (Matzka, 2020). Los cortes se obtienen del entrenamiento y no representan límites industriales.

Figura 3. La interacción de velocidad baja, torque alto y desgaste alto concentra la mayor tasa

La Figura 3 resuelve la pregunta al reunir tres condiciones. Con cortes calculados desde entrenamiento, el segmento completo contiene 101 fallas entre 353 registros y alcanza 28,61 %. El mapa se construyó con una escala común y cifras dentro de cada celda para que la lectura no dependa solo del color (seaborn development team, 2026).

La prueba separada mantiene la diferencia: 28,94 % en entrenamiento y 27,50 % en prueba para el perfil, frente a 2,48 % y 2,40 % en el resto. Esta cercanía fortalece la observación descriptiva. No corresponde a exactitud, sensibilidad ni otra métrica predictiva.

11 Metodología del desarrollo técnico

El cierre siguió seis decisiones: conservar las particiones; recuperar unidades con parámetros aprendidos en entrenamiento; validar las mismas filas contra F1; explorar desde los objetivos; calcular los cortes de interacción solo con entrenamiento; y aplicarlos sin cambios a prueba. Las figuras se organizaron como contexto, contraste y resolución, según los criterios de la guía y el apunte de F4 (Universidad Andrés Bello, 2026a, 2026b).

12 Validación y reflexión técnica

Las verificaciones cubren dimensiones, columnas, nulos, categorías, fallas, equivalencia entre fases, archivos producidos y correspondencia entre objetivos y figuras. El perfil combinado surgió después de la exploración; por eso su comparación en prueba confirma una diferencia descriptiva, pero no constituye una validación de modelo. La principal decisión metodológica fue mantener esa frontera en todas las afirmaciones.

13 Resultados

- La falla aparece en 339 de 10.000 registros, equivalentes al 3,39 %.
- El tipo L alcanza 3,92 %, frente a 2,77 % en M y 2,09 % en H.
- Velocidad baja y torque alto presentan los extremos individuales más marcados.
- El perfil combinado mantiene alrededor de 28 % en entrenamiento y prueba.

14 Discusión

La distribución no es uniforme: torque, velocidad y desgaste separan mejor los segmentos examinados que la temperatura del proceso. Sin embargo, AI4I 2020 es sintético, la clase positiva está desbalanceada y los cuantiles dependen del propio conjunto. Además, la prueba fue inspeccionada para esta confirmación descriptiva. Un estudio predictivo futuro deberá reservar una nueva validación o definir el modelo antes de examinar prueba, y reportar sensibilidad, precisión, F1 y PR-AUC.

15 Conclusiones

El proyecto identificó asociaciones reproducibles entre la ocurrencia de fallas y las condiciones operacionales. La combinación de velocidad baja, torque alto y desgaste elevado define el perfil con mayor frecuencia observada y mantiene una tasa similar en ambas particiones. El objetivo descriptivo queda cumplido con trazabilidad desde F1 hasta F4. La capacidad para anticipar fallas y la generalización a maquinaria real permanecen como trabajo posterior.

16 Trazabilidad de mejoras

Observación	Mejora aplicada	Evidencia
AI4I es sintético y no presenta limpieza real	No se inventaron nulos ni duplicados; se declaró la limitación	F2 y discusión F4
Evitar fuga de información	Se dividió antes de ajustar y se excluyeron los modos de falla	be64efc
Modularizar y medir el código	Se incorporaron clases, Factory, pruebas, timeit y tracemalloc	858ba05
Datos escalados poco interpretables	Se recuperaron unidades y Type con los parámetros de F3	Notebook F4
Sostener el perfil fuera del análisis inicial	Los cortes de entrenamiento se aplicaron sin cambios a prueba	validacion_perfil.csv

Tabla 4. Vinculación entre observaciones, mejoras y evidencia verificable

El archivo changelog.md relaciona las mejoras con commits reales del repositorio. Las huellas de los cuatro archivos recibidos desde F3 quedaron registradas en metadatos_fase4.json.

17 Presentación audiovisual

La presentación se organiza para una duración de 6 a 7 minutos y participación individual. El apoyo visual muestra el problema, la continuidad entre fases, las tres figuras principales, la validación en prueba y los límites. El guion utiliza frases breves para explicar las cifras sin leer el informe ni atribuir capacidad predictiva.

18 Bibliografía

Matzka, S. (2020). AI4I 2020 Predictive Maintenance Dataset [Conjunto de datos]. UCI Machine Learning Repository. <https://doi.org/10.24432/C5HS5C>

pandas development team. (2026). pandas.qcut. pandas documentation. <https://pandas.pydata.org/docs/reference/api/pandas.qcut.html>

Phan, T. L. J., Gehrhardt, I., Heik, D., Bahrpeyma, F., y Reichelt, D. (2022). A systematic mapping study on machine learning techniques applied for condition monitoring and predictive maintenance in the manufacturing sector. Logistics, 6(2), 35. <https://doi.org/10.3390/logistics6020035>

seaborn development team. (2026). seaborn.heatmap. seaborn documentation. <https://seaborn.pydata.org/generated/seaborn.heatmap.html>

Universidad Andrés Bello. (2026a). Guía de apoyo Sumativa 3 Fase 4 [Documento de apoyo docente]. MCDI500 Programación para la Ciencia de Datos.

Universidad Andrés Bello. (2026b). MCDI500 Apunte Fase 4 [Material de clase]. MCDI500 Programación para la Ciencia de Datos.